第一周课后作业标准答案

沈启立，冯赵然

题 1　请简述 P 类问题和 NP 类问题的定义，并解释 “P ⊆NP” 为什么成立。

标准答案：

P 类问题是指可以在多项式时间内求解的判定问题，即存在一个算法，对输入规模为 n 的任意实例，都能在 $O(n^k)$ 时间内求出正确答案，其中 k 为常数。

NP 类问题是指虽然不一定能在多项式时间内直接求解，但给定一个候选解后，可以在多项式时间内验证该解是否正确的判定问题。

之所以

$$P \subseteq NP$$

成立，是因为如果一个问题本身就能在多项式时间内求解，那么给定一个候选答案时，我们只需要重新运行这个多项式时间算法，再将算法输出与候选答案进行比较，就可以在多项式时间内完成验证。因此，所有 P 类问题一定都属于 NP 类问题。

题 2　解释以下概念之间的区别：

(a) 可计算问题与不可计算问题

(b) P 类问题与 NP 类问题

(c) NP-Hard 问题与 NP-Complete 问题

标准答案：

(a) 可计算问题与不可计算问题

可计算问题（decidable / computable）是指：存在某个算法（等价地，某台图灵机），对该问题的**所有输入实例**都能在有限步内停机并给出正确答案。

不可计算问题（undecidable / uncomputable）是指：**不存在**任何算法能够对所有实例都正确求解。

二者的区别在于：可计算问题至少**存在算法**；不可计算问题则连算法都不存在。

(b) P 类问题与 NP 类问题

P 类问题是能在多项式时间内**求解**的问题；NP 类问题是能在多项式时间内**验证**的问题。

二者的关系是

$$P \subseteq NP.$$

区别在于：P 要求直接把答案算出来；NP 只要求如果别人给了一个候选答案，我们能快速检查它是否正确。

(c) NP-Hard 问题与 NP-Complete 问题

NP-Hard 问题是指：**所有 NP 问题都可以多项式时间归约到它**，因此它至少和 NP 中最难的问题一样难。但 NP-Hard 问题**不一定属于 NP**，也就是说它未必是一个“答案可快速验证”的判定问题。

NP-Complete 问题则要求同时满足两点：

1. 它属于 NP；
2. 它是 NP-Hard。

因此，NP-Complete 可以理解为“**NP 中最难的问题**”。所以二者的区别在于：NP-Complete 一定在 NP 中，而 NP-Hard 不一定在 NP 中。NP-Complete 是 NP-Hard 中“还保持可验证性”的那一部分。

题 3 用停机问题（Halting Problem）的不可判定性来解释“优化的上限”。请给出停机问题不可判定的简要证明思路（对角化论证）。

标准答案：

定理 3.1 (图灵, 1936). 不存在图灵机能够判定任意图灵机在任意输入上是否停机。

目标： 证明不存在一个“万能预测器”H，满足

$$H(M,x) = \begin{cases} 1, & M(x) \text{ 会停机} \\ 0, & M(x) \text{ 不会停机} \end{cases}$$

Step 1: 反证法假设
假设这样的图灵机 H 存在，并且预测永远正确。

Step 2: 构造一个“反着来”的机器 D
定义新图灵机 D：

- 输入：图灵机 M 的编码
- 首先运行 $H(M,M)$（让程序以自己为输入）
- 如果 $H(M,M)=1$（预测 $M(M)$ 会停机），则 D 进入死循环
- 如果 $H(M,M)=0$（预测 $M(M)$ 不停机），则 D 立即停机

直观理解： D 总是故意与预测结果相反。

Step 3: 关键问题——考察 $D(D)$
现在让 D 把自己作为输入：$D(D)$。我们分析两种可能。

情况 A: 假设 $D(D)$ 会停机
则根据 H 的定义：$H(D,D)=1$
但根据 D 的规则：若 $H(D,D)=1$，则 $D(D)$ 必须死循环。
矛盾。

情况 B: 假设 $D(D)$ 不会停机
则：$H(D,D)=0$
但根据 D 的规则：若 $H(D,D)=0$，则 $D(D)$ 必须停机。
再次矛盾。

Step 4: 结论
无论 $D(D)$ 是否停机都会产生矛盾。

总结： 优化上限是不可计算的。停机问题是不可判定的。停机问题的不可判定性说明不存在万能判定机。优化的本质是不存在一个对所有程序都安全，自动，且总能最优的机器。

题 4 考虑旅行商问题（TSP），n 个城市。

(a) 暴力枚举算法的时间复杂度是多少？

(b) 当 $n = 15$ 时，暴力枚举大约需要计算多少条不同的路径？

(c) 使用动态规划（Held-Karp 算法）可以将时间复杂度降到多少？

标准答案:

(a) 暴力枚举算法的时间复杂度 若固定一个城市作为起点，则其余 $n-1$ 个城市的访问顺序共有

$$(n-1)!$$

种，因此暴力枚举算法需要检查的路径数量为阶乘级，其时间复杂度为

$$O((n-1)!) = O(n!).$$

若进一步把一条回路的正向与反向视为同一条路径，则无向情形下不同回路数可写为

$$\frac{(n-1)!}{2},$$

但这只影响常数因子，不改变复杂度的数量级，因此暴力枚举的时间复杂度仍为

$$O(n!).$$

(b) 当 $n = 15$ 时，暴力枚举大约需要计算多少条不同的路径 当 $n = 15$ 时，固定起点后，其余 14 个城市的访问顺序共有

$$14! = 87{,}178{,}291{,}200$$

种。

若把正向与反向的巡回路线视为同一条路径，则不同路径数约为

$$\frac{14!}{2} = 43{,}589{,}145{,}600.$$

因此，当 $n = 15$ 时，暴力枚举大约需要计算

$$4.36 \times 10^{10}$$

条不同路径。

(c) Held–Karp 动态规划的时间复杂度 Held–Karp 算法使用状态

$$dp[S][j]$$

表示：从起点出发，恰好访问城市集合 S 中的所有城市，并最终停在城市 j 时的最短路径长度。

其状态转移为

$$dp[S][j] = \min_{i \in S,\ i \neq j} \left(dp[S \setminus \{j\}][i] + d(i, j) \right).$$

其中，$d(i, j)$ 表示城市 i 到城市 j 的距离。

由于：

- 状态总数约为 $n2^n$；
- 每个状态在转移时最多需要枚举 n 个前驱城市；

所以 Held–Karp 算法的总时间复杂度为

$$O(n^2 2^n),$$

空间复杂度为

$$O(n2^n).$$

结论. 暴力枚举算法的时间复杂度为 $O(n!)$；当 $n = 15$ 时，大约需要计算

$$\frac{14!}{2} = 43{,}589{,}145{,}600$$

条不同路径；使用 *Held–Karp* 动态规划后，时间复杂度可降为

$$O(n^2 2^n).$$

Algorithm 1 Held-Karp Algorithm for TSP

Require: Cities $V = \{0, 1, \ldots, n-1\}$
Ensure: Optimal tour and cost

```
Compute distance matrix d(i, j)
for i = 1 to n − 1 do
    DP[{0, i}, i] ← d(0, i)
end for
for r = 3 to n do
    for each subset S ⊆ V with |S| = r and 0 ∈ S do
        for each i ∈ S, i ≠ 0 do
            DP[S, i] ← min_{j∈S, j≠i}
        {DP[S \ {i}, j] + d(j, i)}
        end for
    end for
end for
cost ← min_{i≠0}{DP[V, i] + d(i, 0)}
Reconstruct tour by backtracking
    return tour, cost
```

题 5 什么是多项式时间归约（polynomial-time reduction）？请说明如何将 3-SAT 问题归约到独立集问题（Independent Set），并解释这如何证明独立集问题是 NP-Complete 的。

标准答案：

多项式时间归约是指：

把问题 A 在多项式时间内转化为问题 B。如果 B 能快速解，A 就能快速解；反正，如果 A 很难，则证明 B 至少和 A 一样难。

下面说明如何将 3-SAT 归约到独立集问题。设

$$\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$$

是一个 3-SAT 公式，其中每个子句 C_i 恰好包含三个文字。我们构造一个图 $G = (V, E)$ 以及整数 $k = m$，使得

$$\varphi \text{ 可满足} \iff G \text{ 中存在大小至少为} m \text{ 的独立集.}$$

构造方法如下：

1. 对于每个子句 C_i，建立三个顶点，分别对应这个子句中的三个文字。

2. 在同一个子句内部，将这三个顶点两两连边，因此每个子句对应一个三角形。
3. 对于来自不同子句的两个顶点，如果它们对应的文字互相矛盾，例如一个对应 x_j，另一个对应 $\neg x_j$，则在这两个顶点之间连边。
4. 令目标参数 $k = m$，即询问图中是否存在大小为 m 的独立集。

该构造是多项式时间的，因为若公式有 m 个子句，则顶点数为 $3m$，边也可以在多项式时间内枚举并建立。课件中给出的归约规则正是：每个子句建立一个三角形，三角形内部两两连边，不同子句中互相矛盾的文字之间连边，最后判断是否存在大小等于子句数 m 的独立集。

下面证明该归约的正确性。

（充分性） 若 φ 可满足，则存在一个变量赋值使得每个子句 C_i 至少有一个文字为真。对于每个子句，从对应的三个顶点中选取一个“为真”的文字所对应的顶点。这样一共选出 m 个顶点。由于同一个子句内部形成三角形，因此我们在每个子句中至多选一个点；而由于赋值是一致的，不可能同时选到 x_j 和 $\neg x_j$ 这两个互相矛盾的文字，因此所选顶点之间不会因为“矛盾边”而相邻。于是这 m 个顶点构成一个大小为 m 的独立集。

（必要性） 反过来，若图 G 中存在大小为 m 的独立集 S，由于每个子句内部是一个三角形，而独立集中的任意两个顶点都不能相邻，因此在每个三角形中至多选择一个顶点。又因为总共有 m 个子句，对应 m 个三角形，而独立集大小正好是 m，所以独立集必然在每个三角形中恰好选取一个顶点。于是，每个子句都对应选中了一个文字。又因为独立集中不存在由矛盾文字产生的边，所以不可能同时选到 x_j 和 $\neg x_j$。因此可以据此构造出一个一致的变量赋值，使所有被选中的文字都为真，从而每个子句都为真，所以原公式 φ 可满足。

因此，

$$\varphi \text{ 可满足} \iff G \text{ 中存在大小为} m \text{ 的独立集}.$$

这就完成了 3-SAT 到 Independent Set 的多项式时间归约。

接下来说明这如何证明 Independent Set 是 NP-Complete 的。

首先，Independent Set 属于 NP。因为给定一个顶点集合 S，我们可以在多项式时间内验证：

1. $|S| \geq k$ 是否成立；
2. S 中任意两点之间是否都没有边。

若以上条件成立，则 S 是一个合法的独立集证书，因此 Independent Set $\in$ NP。课件中也给出了 NP 与 NP-Complete 的基本定义：NP 问题是“答案可以快速验证”的问题，而 NP-Complete 问题是 NP 中最难的一类问题。:contentReference[oaicite:6]index=6

其次，3-SAT 是一个已知的 NP-Complete 问题，而我们已经证明了

$$\text{3-SAT} \leq_p \text{Independent Set}.$$

因此，Independent Set 至少和 3-SAT 一样难，即它是 NP-hard 的。

综上，Independent Set 既属于 NP，又是 NP-hard，因此它是 NP-Complete 的。 □

题 6 课程提出“上限–下限”框架：上限是不可计算，下限是暴力枚举 $O(2^n)$。请解释：(a) 为什么说“科研的本质是发现结构、发展针对性方法”？(b) 请用背包问题为例，说明如何通过发现不同层次的结构（如贪心近似、DP 精确解、FPTAS）在上限和下限之间找到好的算法。

标准答案：

1. 首先，理论上限决定了不存在万能优化算法，很多问题在本质上是不可计算的（例如停机问题），或者极其困难（例如 NP-Complete）。因此我们必须放弃寻找银弹的执念，承认每个问题都有其特定的本质难度；其次，暴力下限在工程实践中几乎完全不可行。盲目地随机猜测或者是进行时间复杂度为 $O(2^n)$ 甚至 $O(n!)$ 的暴力枚举会让计算量发生指数级爆炸，在实际工程中无落地可能；最后，理论上的一般性困难并不意味着实际应用中的问题无法处理，因为现实问题往往天然蕴含特殊结构。科研的本质在于识别具体问题特有的内部结构，开发量身定制的算法，从而在最坏情况的理论极值、实际可接受的误差以及现代计算硬件（如 GPU 并行范式）之间，找到最优的平衡点。

2. 背包问题（Knapsack problem）：给定一组物品，每种物品都有自己的重量和价格，在限定的总重量内，我们如何选择，才能使得物品的总价格最高。
   给定物品 $x_i, i = 1, 2, ..., n$，重量为 w_i，价值为 v_i，背包容量 W，目标价值 V。
   问：$\exists x_i \in \{0, 1\}$
   满足
   $\sum w_i x_i \leq W$
   $\sum v_i x_i \geq V$
   （1）对于有 n 个物品的背包问题，如果仅做暴力枚举，那么复杂度为 $O(2^n)$，哪怕只有 50 个物品，计算量也会达到 10^{15} 级别，实际工程中完全不可行。
   （2）发现“单位价值”结构 → 贪心算法：在问题 setting 中，物品同时拥有价值和

重量，我们可以通 / 得到“性价比”，随后依次把性价比最高的物品塞入背包，直至装不下为止。此时复杂度会降低到 $O(n \log n)$ 级别。这种算法追求计算效率，但很有可能会浪费背包空间，陷入局部最优。但它至少可以在较短时间内给出尚可接受的解。

（3）发现“子问题重叠”结构 → DP：如果我们仔细观察选择过程，会发现一个重叠子结构：当我们在决定是否放入第 i 个物品时，其实是在求解一个缩小版的原问题——“用剩余的背包容量，在前 $i-1$ 个物品中选出最大价值”。因此可以构建一个表格 $DP[i][W]$ 记录“在前 i 个物品中，背包容量为 W 时的最大价值”。通过记忆表格，避免了 $O(2^n)$ 的重复搜索，将时间复杂度降低到了 $O(n \times W)$ 级别。这可以实现绝对精确的最优解，但如果背包容量 W 较大，这个算法仍然面临难以落地的问题。

（4）发现“数值精确度与复杂度的权衡”结构 → FPTAS：在 DP 算法中，导致表格过大的是庞大的容量 W 或极高的物品价值。但现实工程中，我们往往不需要精确到小数点后哪怕一分钱的差异。如果我们将所有物品的价值都按比例缩小并抹零（取整），例如将价值 10005 缩小为 10，状态空间的表格就会急剧缩小。这会引入一个在接受范围内的误差 ϵ，在理论和工程中找到了平衡。它时间复杂度变为 $O(n^3/\epsilon)$，仅牺牲了极小且可控的精度，实现了这个 NP-Complete 问题真正意义上的多项式时间求解。

Algorithm 2 FPTAS for Knapsack Problem

Require: Items $\{1, 2, \ldots, n\}$, weights w_i, values v_i, capacity W, error tolerance $\epsilon \in (0, 1)$
Ensure: Approximate subset S and its original total value

```
v_max ← max_{1≤i≤n} v_i
K ← ε·v_max / n                          ▷ Calculate the scaling factor
for i = 1 to n do
    v'_i ← ⌊v_i / K⌋                     ▷ Scale and round down the values
end for
V_max ← Σ_{i=1}^{n} v'_i                 ▷ Upper bound of the scaled values
Initialize DP[0 ... V_max] with ∞
DP ← 0
for i = 1 to n do
    for v = V_max down to v'_i do
        DP[v] ← min(DP[v], DP[v − v'_i] + w_i)
    end for
end for
v* ← max{v | DP[v] ≤ W}                  ▷ Find max scaled value within capacity
Reconstruct subset S by backtracking the DP table
total_value ← Σ_{i∈S} v_i                ▷ Return the sum of original values
    return S, total_value
```

题 7 GPU 并行可以改变我们对复杂度的看法。假设有一个 $O(n^2)$ 的算法 A 可以完全并行化（每步独立），和一个 $O(n\log n)$ 的算法 B 本质上是顺序的。

(a) 在拥有 p 个并行处理器的情况下，算法 A 的并行时间复杂度是多少？

(b) 在什么条件下（n 和 p 的关系），并行化的 A 比顺序的 B 更快？

(c) 这对“最优算法”的概念有何启示？

标准答案：

(a) 在拥有 p 个并行处理器的情况下，算法 A 的并行时间复杂度 算法 A 的总工作量为

$$O(n^2).$$

若它可以被完全并行化，并且这 p 个处理器能够理想地平均分担工作量，则每个处理器只需承担原来约 $1/p$ 的计算量，因此算法 A 的并行时间复杂度为

$$O\left(\frac{n^2}{p}\right).$$

这里默认忽略通信、同步以及负载不均衡等额外开销。

(b) 在什么条件下，并行化的 A 比顺序的 B 更快 算法 B 的顺序时间复杂度为

$$O(n\log n).$$

若希望并行化后的算法 A 比算法 B 更快，则需要满足

$$\frac{n^2}{p} < n\log n.$$

两边同时除以 n，得到

$$\frac{n}{p} < \log n.$$

进一步化简为

$$p > \frac{n}{\log n}.$$

因此，当处理器数量 p 足够大，满足

$$p > \frac{n}{\log n},$$

则并行化的算法 A 可能比顺序的算法 B 更快。

若把常数因子也写出来，设

$$T_A = c_1 \frac{n^2}{p}, \qquad T_B = c_2 n \log n,$$

则更一般的条件为

$$p > \frac{c_1}{c_2} \cdot \frac{n}{\log n}.$$

(c) 这对“最优算法”概念的启示 这说明“最优算法”并不是脱离计算模型而绝对存在的概念，而是与具体硬件和执行方式密切相关。

在传统串行计算模型下，通常认为

$$O(n \log n)$$

优于

$$O(n^2).$$

但在并行计算模型下，一个总工作量更大、却能够高度并行化的算法，可能拥有更短的实际运行时间。因此，现代算法设计不能只比较串行渐近复杂度，还应同时考虑：

- 算法的可并行性；
- 并行处理器数量；
- 通信与同步开销；
- 内存访问模式与目标硬件特性。

结论. 在拥有 p 个并行处理器时，算法 A 的并行时间复杂度为

$$O\left(\frac{n^2}{p}\right).$$

当

$$p > \frac{n}{\log n}$$

时，并行化的 A 可能比顺序的 $O(n \log n)$ 算法 B 更快。这说明“最优算法”依赖于计算模型，在并行硬件上，复杂度更高但并行性更强的算法也可能更优。

题 8 图着色到 3-SAT 的归约

具体题目：

考虑图 G 有 3 个节点 $\{v_1, v_2, v_3\}$ 和 3 条边

$$\{(v_1, v_2), (v_2, v_3), (v_1, v_3)\},$$

即完全图 K_3。使用 3 种颜色 $\{R, G, B\}$ 的图着色问题。

将 3-着色问题编码为 SAT：为每个节点 v_i 和颜色 c 引入布尔变量 $x_{i,c}$，表示“节点 v_i 被染成颜色 c”。

(a) 至少一种颜色：为每个节点写一个子句，保证它至少被染一种颜色。例如 v_1 对应

$$(x_{1,R} \vee x_{1,G} \vee x_{1,B}).$$

写出全部 3 个子句。

(b) 至多一种颜色：对每个节点，任意两种颜色不能同时为真。例如 v_1 不能同时为 R 和 G：

$$(\neg x_{1,R} \vee \neg x_{1,G}).$$

写出全部

$$3 \times \binom{3}{2} = 9$$

个子句。

(c) 相邻不同色：对每条边和每种颜色，两个端点不能同色。例如边 (v_1, v_2) 在颜色 R 下对应

$$(\neg x_{1,R} \vee \neg x_{2,R}).$$

写出全部

$$3 \times 3 = 9$$

个子句。

(d) 共多少个变量？多少个子句？手动求解该 SAT 实例（找到所有满足赋值），验证它们恰好对应 K_3 的所有合法 3-着色方案。K_3 有多少种合法着色？

(e) 如果改为 K_4（4 个节点的完全图，仍用 3 种颜色），变量数和子句数分别变为多少？该 SAT 实例是否可满足？为什么？

标准答案：

(a) 至少一种颜色

为每个节点写一个子句，保证它至少被染一种颜色。例如 v_1 对应 $(x_{1,R} \vee x_{1,G} \vee x_{1,B})$。写出全部 3 个子句。

$$(x_{1,R} \vee x_{1,G} \vee x_{1,B})$$

$$(x_{2,R} \vee x_{2,G} \vee x_{2,B})$$

$$(x_{3,R} \vee x_{3,G} \vee x_{3,B})$$

(b) 至多一种颜色

对每个节点，任意两种颜色不能同时为真。例如 v_1 不能同时为 R 和 G：$(\neg x_{1,R} \vee \neg x_{1,G})$。写出全部 $3 \times \binom{3}{2} = 9$ 个子句。

$$(\neg x_{1,R} \vee \neg x_{1,G}), \quad (\neg x_{1,R} \vee \neg x_{1,B}), \quad (\neg x_{1,G} \vee \neg x_{1,B})$$

$$(\neg x_{2,R} \vee \neg x_{2,G}), \quad (\neg x_{2,R} \vee \neg x_{2,B}), \quad (\neg x_{2,G} \vee \neg x_{2,B})$$

$$(\neg x_{3,R} \vee \neg x_{3,G}), \quad (\neg x_{3,R} \vee \neg x_{3,B}), \quad (\neg x_{3,G} \vee \neg x_{3,B})$$

(c) 相邻不同色

对每条边和每种颜色，两个端点不能同色。例如边 (v_1, v_2) 颜色 R：$(\neg x_{1,R} \vee \neg x_{2,R})$。写出全部 $3 \times 3 = 9$ 个子句。

$$(\neg x_{1,R} \vee \neg x_{2,R}), \quad (\neg x_{1,G} \vee \neg x_{2,G}), \quad (\neg x_{1,B} \vee \neg x_{2,B})$$

$$(\neg x_{2,R} \vee \neg x_{3,R}), \quad (\neg x_{2,G} \vee \neg x_{3,G}), \quad (\neg x_{2,B} \vee \neg x_{3,B})$$

$$(\neg x_{1,R} \vee \neg x_{3,R}), \quad (\neg x_{1,G} \vee \neg x_{3,G}), \quad (\neg x_{1,B} \vee \neg x_{3,B})$$

(d) 共多少个变量？多少个子句？

手动求解该 SAT 实例（找到所有满足赋值），验证它们恰好对应 K_3 的所有合法 3-着色方案。K_3 有多少种合法着色?

解答：

- 变量数：$3 \times 3 = 9$ 个变量。
- 子句数：$3 + 9 + 9 = 21$ 个子句。

- 方案数：K_3 共有 6 种合法着色。

对应的满足赋值（满足方案）如下：

1. $v_1 = R, v_2 = G, v_3 = B \implies x_{1,R} = x_{2,G} = x_{3,B} = \text{True}$
2. $v_1 = G, v_2 = R, v_3 = B \implies x_{1,G} = x_{2,R} = x_{3,B} = \text{True}$
3. $v_1 = B, v_2 = R, v_3 = G \implies x_{1,B} = x_{2,R} = x_{3,G} = \text{True}$
4. $v_1 = R, v_2 = B, v_3 = G \implies x_{1,R} = x_{2,B} = x_{3,G} = \text{True}$
5. $v_1 = B, v_2 = G, v_3 = R \implies x_{1,B} = x_{2,G} = x_{3,R} = \text{True}$
6. $v_1 = G, v_2 = B, v_3 = R \implies x_{1,G} = x_{2,B} = x_{3,R} = \text{True}$

(e) K_4 图的情况

如果改为 K_4（4 个节点的完全图，仍用 3 种颜色），变量数和子句数分别变为多少？该 SAT 实例是否可满足？为什么？

解答：

- 变量数变为：$4 \times 3 = 12$ 个变量。
- 子句数变为：$4 + 12 + 18 = 34$ 个子句。

该 SAT 实例不满足。因为 K_4 是 4 个节点两两相邻的完全图。如果要合法 3-着色，那么任意两个节点都不能同色。

但：

- 4 个两两相邻的点
- 需要两两不同色
- 至少需要 4 种颜色

而现在只有 3 种颜色 R, G, B，根据抽屉原理，4 个点里至少有两个点同色。但在 K_4 中任意两个点都相邻，所以这会违反“相邻不同色”。

题 9　归约手算：子集和 $\to$ SAT

具体题目：

子集和问题：给定集合

$$S = \{3, 5, 7\}$$

和目标

$$T = 8,$$

问是否存在一个子集，其元素和恰好为 T。

用布尔变量 x_i 表示是否选择第 i 个元素，其中

$$x_i = 1$$

表示选择第 i 个元素，$x_i = 0$ 表示不选。

(a) 写出对应的整数约束：

$$3x_1 + 5x_2 + 7x_3 = 8.$$

枚举所有 $2^3 = 8$ 种赋值，并指出哪些赋值满足该约束。

(b) 将整数约束转化为二进制表示的逻辑电路（加法器），再将电路转化为 CNF 子句（Tseitin 变换）。画出

$$3x_1 + 5x_2$$

的二进制加法电路，其中

$$3 = 011_2, \qquad 5 = 101_2,$$

和最多为

$$1000_2.$$

(c) 对一个全加器（3 输入 1 位加法）

$$a + b + c_{in} = 2c_{out} + s,$$

写出输出位 s 和进位 c_{out} 的逻辑表达式，并进一步转化为 CNF（可采用 Tseitin 变换，引入中间变量）。

(d) 这个归约说明了什么？（子集和是 NP-Complete，因此它可以被编码为 SAT。）

标准答案：

(a) $3x_1 + 5x_2 + 7x_3$ 的真值表

表 1 给出了 (x_1, x_2, x_3) 的所有可能赋值及对应的 $3x_1 + 5x_2 + 7x_3$ 的值。使结果为 8 的唯一赋值为 $(x_1, x_2, x_3) = (1, 1, 0)$。

(x_1, x_2, x_3)	$3x_1 + 5x_2 + 7x_3$
$(0,0,0)$	0
$(0,0,1)$	7
$(0,1,0)$	5
$(1,0,0)$	3
$(1,1,0)$	8
$(1,0,1)$	10
$(0,1,1)$	12
$(1,1,1)$	15

表 1: $3x_1 + 5x_2 + 7x_3$ 的真值表。

(b) $3x_1 + 5x_2$ 的 CNF 编码

现在构造一个 CNF 公式，它在位级上描述 $3x_1 + 5x_2$ 的计算。将 3 和 5 写成二进制：$3 = 011_2$，$5 = 101_2$，于是

$$3x_1 = (0,\ x_1,\ x_1)_2, \qquad 5x_2 = (x_2,\ 0,\ x_2)_2.$$

将这两个 3 位数从第 0 位到第 2 位相加，结果最多为 4 位（1000_2）。引入中间进位变量 c_1, c_2, c_3 和和位变量 s_0, s_1, s_2, s_3。每个逻辑门都被转化为等价的 CNF 子句。

9.2.1 第 0 位（权重 2^0）

输入：x_1, x_2。半加器的逻辑表达式如下：

和：$s_0 = x_1 \oplus x_2$，对应的合取范式为：

$$(\neg x_1 \vee \neg x_2 \vee \neg s_0) \wedge (x_1 \vee x_2 \vee \neg s_0) \wedge (x_1 \vee \neg x_2 \vee s_0) \wedge (\neg x_1 \vee x_2 \vee s_0),$$

进位：$c_1 = x_1 \wedge x_2$，对应的合取范式为：

$$(\neg c_1 \vee x_1) \wedge (\neg c_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2 \vee c_1).$$

9.2.2 第 1 位（权重 2^1）

输入：x_1、常数 0 以及来自上一位的进位 c_1。实际上只需一个半加器计算 $x_1 + c_1$：

和：$s_1 = x_1 \oplus c_1$，对应的合取范式为：

$$(\neg x_1 \vee \neg c_1 \vee \neg s_1) \wedge (x_1 \vee c_1 \vee \neg s_1) \wedge (x_1 \vee \neg c_1 \vee s_1) \wedge (\neg x_1 \vee c_1 \vee s_1),$$

进位：$c_2 = x_1 \wedge c_1$，对应的合取范式为：

$$(\neg c_2 \vee x_1) \wedge (\neg c_2 \vee c_1) \wedge (\neg x_1 \vee \neg c_1 \vee c_2).$$

9.2.3 第 2 位（权重 2^2）

输入：常数 0、x_2 以及进位 c_2。半加器计算 $x_2 + c_2$：

和：$s_2 = x_2 \oplus c_2$，对应的合取范式为：

$$(\neg x_2 \vee \neg c_2 \vee \neg s_2) \wedge (x_2 \vee c_2 \vee \neg s_2) \wedge (x_2 \vee \neg c_2 \vee s_2) \wedge (\neg x_2 \vee c_2 \vee s_2),$$

进位：$c_3 = x_2 \wedge c_2$，对应的合取范式为：

$$(\neg c_3 \vee x_2) \wedge (\neg c_3 \vee c_2) \wedge (\neg x_2 \vee \neg c_2 \vee c_3).$$

9.2.4 第 3 位（权重 2^3）

最高和位就是最后的进位：

和：$s_3 = c_3$，对应的合取范式为：

$$(\neg s_3 \vee c_3) \wedge (\neg c_3 \vee s_3).$$

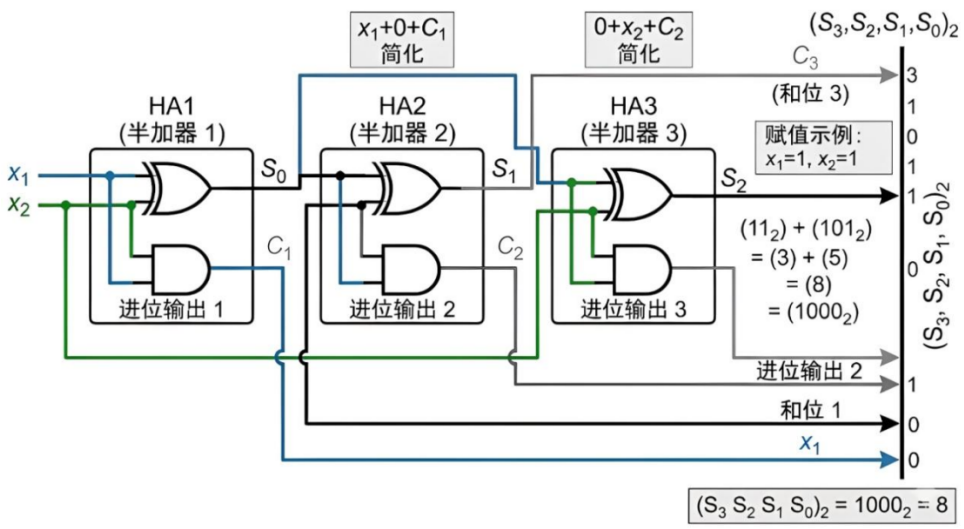
9.2.5 完整的 CNF 公式

$3x_1 + 5x_2$ 的二进制表示为 4 位输出 $(s_3, s_2, s_1, s_0)_2$。将上述所有子句合取，即得到描述该电路的 CNF 公式：

$$\begin{aligned}
&(x_1 \vee x_2 \vee \neg s_0) \wedge (x_1 \vee \neg x_2 \vee s_0) \wedge (\neg x_1 \vee x_2 \vee s_0) \wedge (\neg x_1 \vee \neg x_2 \vee \neg s_0) \\
&\wedge (\neg x_1 \vee \neg x_2 \vee c_1) \wedge (x_1 \vee \neg c_1) \wedge (x_2 \vee \neg c_1) \\
&\wedge (x_1 \vee c_1 \vee \neg s_1) \wedge (x_1 \vee \neg c_1 \vee s_1) \wedge (\neg x_1 \vee c_1 \vee s_1) \wedge (\neg x_1 \vee \neg c_1 \vee \neg s_1) \\
&\wedge (\neg x_1 \vee \neg c_1 \vee c_2) \wedge (x_1 \vee \neg c_2) \wedge (c_1 \vee \neg c_2) \\
&\wedge (x_2 \vee c_2 \vee \neg s_2) \wedge (x_2 \vee \neg c_2 \vee s_2) \wedge (\neg x_2 \vee c_2 \vee s_2) \wedge (\neg x_2 \vee \neg c_2 \vee \neg s_2) \\
&\wedge (\neg x_2 \vee \neg c_2 \vee c_3) \wedge (x_2 \vee \neg c_3) \wedge (c_2 \vee \neg c_3) \\
&\wedge (\neg s_3 \vee c_3) \wedge (s_3 \vee \neg c_3).
\end{aligned}$$

9.2.6 画出二进制加法电路

$3x_1 + 5x_2$ 的二进制加法电路实际上就是三个半加器串联起来：



(c) 全加器的逻辑表达式与 CNF 编码

对于一个全加器（输入 a, b, c_{in}，输出 s, c_{out}），满足加法规则：

$$a + b + c_{\text{in}} = 2c_{\text{out}} + s$$

其逻辑表达式为：

- 和 (Sum)：$s = a \oplus b \oplus c_{\text{in}}$
- 进位 (Carry)：$c_{\text{out}} = (a \wedge b) \vee (c_{\text{in}} \wedge (a \oplus b))$

s 的 CNF（4 个三元子句）：

$$(a \vee b \vee c_{\text{in}}) \wedge (a \vee \neg b \vee \neg c_{\text{in}}) \wedge (\neg a \vee b \vee \neg c_{\text{in}}) \wedge (\neg a \vee \neg b \vee c_{\text{in}})$$

Tseitin 变换 为了将电路转换为 CNF（合取范式），为每个基本逻辑门的输出引入中间变量，并为每个门写出等价的局部 CNF 子句集合。c_{out} 的 CNF 分解（引入中间变量）：

$$x \leftrightarrow a \oplus b$$
$$g \leftrightarrow a \wedge b$$
$$p \leftrightarrow c_{\text{in}} \wedge x$$
$$c_{\text{out}} \leftrightarrow g \vee p$$

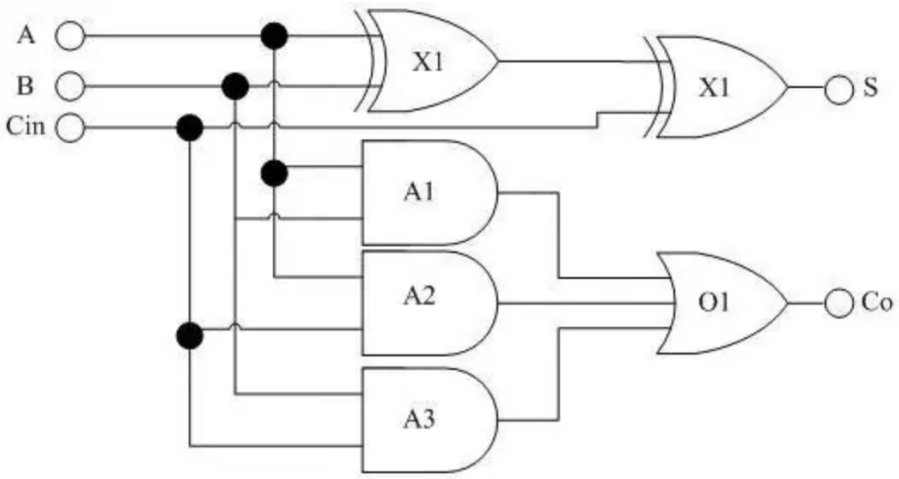
各中间变量等价的 CNF：

- $g \leftrightarrow a \wedge b$ 的 CNF：
  $(\neg g \vee a) \wedge (\neg g \vee b) \wedge (g \vee \neg a \vee \neg b)$
- $p \leftrightarrow c_{\text{in}} \wedge x$ 的 CNF：
  $(\neg p \vee c_{\text{in}}) \wedge (\neg p \vee x) \wedge (\neg c_{\text{in}} \vee p \vee \neg x)$
- $c_{\text{out}} \leftrightarrow g \vee p$ 的 CNF：
  $(\neg g \vee c_{\text{out}}) \wedge (\neg p \vee c_{\text{out}}) \wedge (\neg c_{\text{out}} \vee g \vee p)$

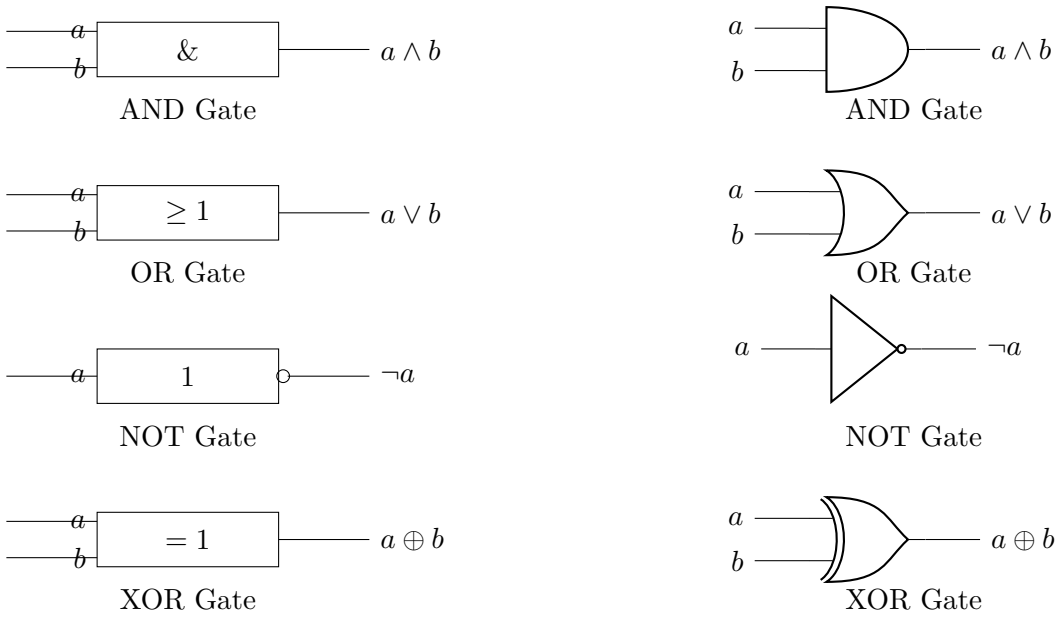
全加器的逻辑电路简化表达式：

$$\begin{cases} S = A \oplus B \oplus C_{\text{in}} \\ C_{\text{out}} = AB + BC + CA \end{cases}$$

全加器的逻辑电路可表示为：



补充：逻辑电路的画法：左侧为国标图形符号，右侧为 ANSI 图形符号



(d) 子集和问题与 SAT 的归约

子集和问题是一个经典的 NP-Complete 问题，可以在多项式时间内编码为布尔可满足性问题（SAT）。通过将整数加法转化为二进制加法电路，再将电路转化为 CNF 公式，使得原问题有解当且仅当该 CNF 公式可满足。

这体现了 SAT 的通用性，即任何 NP 问题都可归约到 SAT，从而 SAT 是 NP-Complete 的。同时，它也提供了一种用 SAT 求解器解决子集和问题的途径：** 数值计算可以被逻辑公式编码 **，通过求解逻辑公式的可满足性，即可等价求解原子集和问题。

题 10　SAT 求解器体验

具体题目：

用 Python 实现一个暴力 SAT 求解器：输入一个 CNF 公式（用子句列表表示），枚举所有 2^n 种变量赋值，并检查该赋值是否满足公式。

(a) 对以下 CNF 公式求解：

$$(x_1 \vee x_2) \ \wedge \ (\neg x_1 \vee x_3) \ \wedge \ (\neg x_2 \vee \neg x_3).$$

(b) 随机生成含 n 个变量、m 个子句（每个子句包含 3 个文字）的 3-SAT 实例，测试当

$$n = 10,\ 15,\ 20$$

时的求解时间。

(c) 固定

$$n = 20,$$

画出可满足概率随 m/n 比值变化的曲线（字句密度相变现象）

标准答案:

(a) 准备工具箱

首先实现暴力 SAT 求解器和随机 3-SAT 实例生成器:

```python
import itertools
import time
import random
import matplotlib.pyplot as plt
import numpy as np

def solve_sat_brute_force(num_vars, clauses):
    """暴力 SAT 求解器函数"""
    for assignment in itertools.product([False, True], repeat=num_vars):
        x = {i + 1: assignment[i] for i in range(num_vars)}
        all_satisfied = True
        for clause in clauses:
            clause_satisfied = False
            for literal in clause:
                var_idx = abs(literal)
                val = x[var_idx]
                if literal < 0: val = not val
                if val:
                    clause_satisfied = True
                    break
            if not clause_satisfied:
                all_satisfied = False
                break
        if all_satisfied:
            return x
    return None

def generate_random_3sat(n, m):
    """随机生成 3-SAT 实例"""
    clauses = []
```

```
    for _ in range(m):
        vars_picked = random.sample(range(1, n + 1), 3)
        clause = []
        for v in vars_picked:
            clause.append(v if random.random() < 0.5 else -v)
        clauses.append(clause)
    return clauses
```

(b) 第一问：对公式 $(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$ 求解

规则是：$(x_1$ 或 $x_2)$ 且 $(\neg x_1$ 或 $x_3)$ 且 $(\neg x_2$ 或 $\neg x_3)$。

```
clauses_a = [[1, 2], [-1, 3], [-2, -3]]
solution_a = solve_sat_brute_force(3, clauses_a)
print("第一问的答案是：", solution_a)
```

运行结果：

```
第一问的答案是： {1: False, 2: True, 3: False}
```

(c) 第二问：测试速度

```
for n in [10, 15, 20]:
    m = int(4.2 * n)
    test_clauses = generate_random_3sat(n, m)
    start = time.time()
    solve_sat_brute_force(n, test_clauses)
    print(f"开关数 n={n}, 耗时：{time.time() - start:.4f} 秒")
```

运行结果：

```
开关数 n=10, 耗时：0.0003 秒
开关数 n=15, 耗时：0.0242 秒
开关数 n=20, 耗时：1.0782 秒
```

(d) 第三问：可满足概率的相变现象 $n = 20$，画出“可满足概率”随

```
n_fixed = 20
ratios = np.linspace(3, 6, 10)
probs = []

```

```python
print("正在为您计算，请稍等片刻...")
for r in ratios:
    m = int(r * n_fixed)
    sat_count = 0
    for _ in range(10):
        c = generate_random_3sat(n_fixed, m)
        if solve_sat_brute_force(n_fixed, c):
            sat_count += 1
    probs.append(sat_count / 10)

plt.plot(ratios, probs, 'o-')
plt.xlabel("规则密度 (m/n)")
plt.ylabel("概率")
plt.title(f"SAT 可满足概率的变化 ({n_fixed}个开关)")
plt.grid(True)
plt.show()
```

运行结果如图所示：

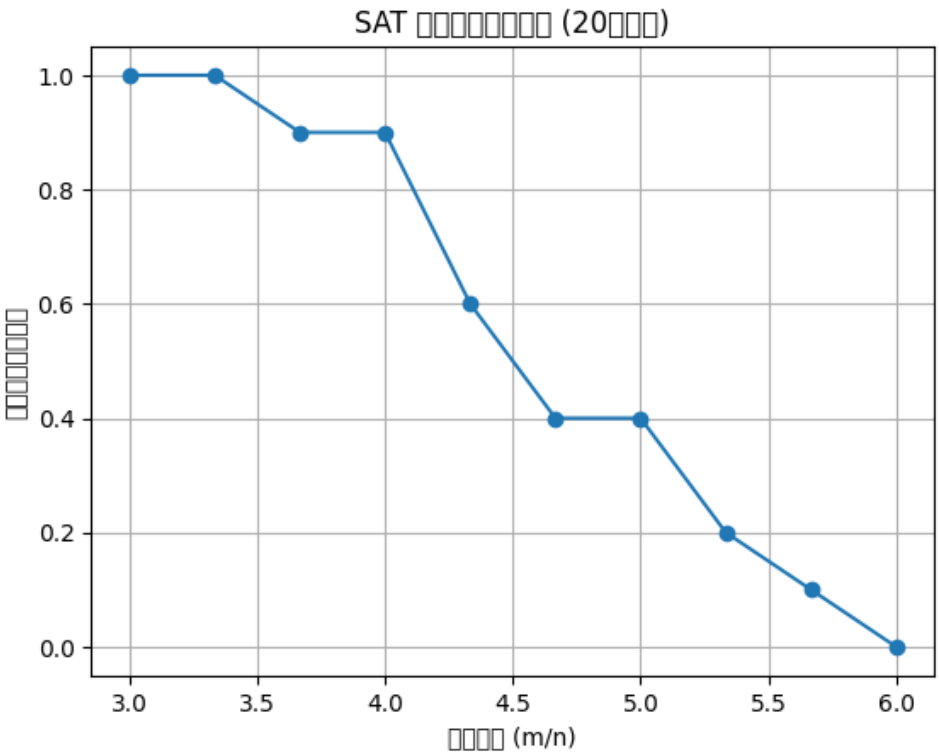


图 1: 固定 $n = 20$，可满足概率随子句密度 m/n 变化的曲线

从图中可以观察到典型的**相变现象**：当子句密度 m/n 较小时，约束较少，实例大多可满足，概率接近 1；随着 m/n 增大，可满足概率在某个临界密度（约 $m/n \approx 4.2$ 附

近）陡然下降；当 m/n 足够大时，概率趋近于 0。这条 S 型陡降曲线体现了随机 3-SAT 的相变现象，也呼应了课程中的核心观点：SAT 属于 NP-Complete，验证一个赋值很快，但求解本身随规模迅速变难。